

```

In [ ]: import tkinter as tk
        from tkinter import ttk, messagebox
        from datetime import datetime
        import threading

        tasks = []

        def add_task():
            description = entry.get()
            due_date_str = due_date_entry.get()
            try:
                due_date = datetime.strptime(due_date_str, '%Y-%m-%d %H:%M')
            except ValueError:
                messagebox.showwarning("Warning", "Due date must be in 'YYYY-MM-DD HH:MM' format")
                return

            if description:
                task_id = len(tasks) + 1
                tasks.append({'id': task_id, 'description': description, 'status': 'incomplete', 'due_date': due_date})
                update_task_listbox()
                entry.delete(0, tk.END)
                due_date_entry.delete(0, tk.END)
            else:
                messagebox.showwarning("Warning", "Task description cannot be empty")

        def update_task():
            try:
                selected = listbox.selection()[0]
                task_id = int(listbox.item(selected)['values'][0])
                new_description = entry.get()
                due_date_str = due_date_entry.get()

                if new_description and due_date_str:
                    try:
                        new_due_date = datetime.strptime(due_date_str, '%Y-%m-%d %H:%M')
                    except ValueError:
                        messagebox.showwarning("Warning", "Due date must be in 'YYYY-MM-DD HH:MM' format")
                        return

                    for task in tasks:
                        if task['id'] == task_id:
                            task['description'] = new_description

```

```

        task['due_date'] = new_due_date
        break
    update_task_listbox()
    entry.delete(0, tk.END)
    due_date_entry.delete(0, tk.END)
else:
    messagebox.showwarning("Warning", "Task description and due date cannot be empty")
except IndexError:
    messagebox.showwarning("Warning", "No task selected")

def delete_task():
    try:
        selected = listbox.selection()[0]
        task_id = int(listbox.item(selected)['values'][0])
        global tasks
        tasks = [task for task in tasks if task['id'] != task_id]
        update_task_listbox()
    except IndexError:
        messagebox.showwarning("Warning", "No task selected")

def mark_task(status):
    try:
        selected = listbox.selection()[0]
        task_id = int(listbox.item(selected)['values'][0])
        for task in tasks:
            if task['id'] == task_id:
                task['status'] = status
                break
        update_task_listbox()
    except IndexError:
        messagebox.showwarning("Warning", "No task selected")

def update_task_listbox():
    for i in listbox.get_children():
        listbox.delete(i)
    for task in tasks:
        status = '✓' if task['status'] == 'complete' else 'X'
        due_date_str = task['due_date'].strftime('%Y-%m-%d %H:%M')
        listbox.insert('', 'end', values=(task['id'], task['description'], status, due_date_str))

def filter_tasks(status=None):
    for i in listbox.get_children():
        listbox.delete(i)
    for task in tasks:

```

```

        if status is None or task['status'] == status:
            status_mark = '✓' if task['status'] == 'complete' else 'X'
            due_date_str = task['due_date'].strftime('%Y-%m-%d %H:%M')
            listbox.insert('', 'end', values=(task['id'], task['description'], status_mark, due_date_str))

def check_due_tasks():
    while True:
        now = datetime.now()
        for task in tasks:
            if task['status'] == 'incomplete' and task['due_date'] <= now:
                messagebox.showinfo("Task Reminder", f"Task '{task['description']}' is due!")
                task['status'] = 'complete'
                update_task_listbox()
        threading.Event().wait(60) # Check every 60 seconds

app = tk.Tk()
app.title("To-Do List")
app.geometry("600x400")

style = ttk.Style(app)
style.theme_use('clam')
style.configure("Treeview", background="white", foreground="black", rowheight=25, fieldbackground="white")
style.map('Treeview', background=[('selected', '#347083')])

frame = ttk.Frame(app)
frame.pack(pady=10)

entry = ttk.Entry(frame, width=30)
entry.pack(side=tk.LEFT, padx=5)

due_date_entry = ttk.Entry(frame, width=20)
due_date_entry.pack(side=tk.LEFT, padx=5)
due_date_entry.insert(0, 'YYYY-MM-DD HH:MM')

add_button = ttk.Button(frame, text="Add Task", command=add_task)
add_button.pack(side=tk.LEFT)

columns = ('ID', 'Description', 'Status', 'Due Date')
listbox = ttk.Treeview(app, columns=columns, show='headings')
listbox.heading('ID', text='ID')
listbox.heading('Description', text='Description')
listbox.heading('Status', text='Status')
listbox.heading('Due Date', text='Due Date')
listbox.column('ID', width=50)

```

```
listbox.column('Description', width=250)
listbox.column('Status', width=50)
listbox.column('Due Date', width=150)
listbox.pack(pady=20)

btn_frame = ttk.Frame(app)
btn_frame.pack(pady=10)

update_button = ttk.Button(btn_frame, text="Update Task", command=update_task)
update_button.grid(row=0, column=0, padx=5)

delete_button = ttk.Button(btn_frame, text="Delete Task", command=delete_task)
delete_button.grid(row=0, column=1, padx=5)

complete_button = ttk.Button(btn_frame, text="Mark Complete", command=lambda: mark_task('complete'))
complete_button.grid(row=0, column=2, padx=5)

incomplete_button = ttk.Button(btn_frame, text="Mark Incomplete", command=lambda: mark_task('incomplete'))
incomplete_button.grid(row=0, column=3, padx=5)

filter_frame = ttk.Frame(app)
filter_frame.pack(pady=10)

all_button = ttk.Button(filter_frame, text="All", command=lambda: filter_tasks(None))
all_button.grid(row=0, column=0, padx=5)

completed_button = ttk.Button(filter_frame, text="Completed", command=lambda: filter_tasks('complete'))
completed_button.grid(row=0, column=1, padx=5)

incomplete_button = ttk.Button(filter_frame, text="Incomplete", command=lambda: filter_tasks('incomplete'))
incomplete_button.grid(row=0, column=2, padx=5)

# Start the background thread to check for due tasks
thread = threading.Thread(target=check_due_tasks, daemon=True)
thread.start()

app.mainloop()
```

In [3]:

In []: